ORACLE
Academy

Database Programming with PL/SQL

13-2
Creating DML Triggers: Part I

ORACLE
Academy

Objectives

- This lesson covers the following objectives:
  - Create a DML trigger
  - List the DML trigger components
  - Create a statement-level trigger
  - Describe the trigger firing sequence options

Purpose

- Suppose you want to keep an automatic record of the history of changes to employees’ salaries
- This is not only important for business reasons, but is a legal requirement in many countries
- To do this, you create a DML trigger
- DML triggers are the most common type of trigger in most Oracle databases
- In this and the next lesson, you learn how to create and use database DML triggers

What Is a DML Trigger?

- A DML trigger is a trigger that is automatically fired (executed) whenever an SQL DML statement (INSERT, UPDATE, or DELETE) is executed
- You classify DML triggers in two ways:
  - By when they execute: BEFORE, AFTER, or INSTEAD OF the triggering DML statement
  - By how many times they execute: Once for the whole DML statement (a statement trigger), or once for each row affected by the DML statement (a row trigger)

ORACLE Academy

In this lesson, you learn about statement triggers. The next lesson will cover row triggers.

NOTE: You cannot explicitly create a trigger on a MERGE DML statement. However, since a MERGE is really a mixture of INSERTs and UPDATEs, creating INSERT and/or UPDATE triggers will produce the same effect.

Creating DML Statement Triggers

- The sections of a CREATE TRIGGER statement that need to be considered before creating a trigger:

```
CREATE [OR REPLACE] TRIGGER trigger_name
  timing
  event1 [OR event2 OR event3] ON object_name
trigger_body
```

  - timing: When the trigger fires in relation to the triggering event
    - Values are BEFORE, AFTER, or INSTEAD OF
  - event: Which DML operation causes the trigger to fire
  - Values are INSERT, UPDATE [OF column], and DELETE

Creating DML Statement Triggers

```
CREATE [OR REPLACE] TRIGGER trigger_name
  timing
  event1 [OR event2 OR event3] ON object_name
trigger_body
```

- object_name: The table or view associated with the trigger
- trigger_body: The action(s) performed by the trigger are defined in an anonymous block

Statement Trigger Timing

- When should the trigger fire?
- BEFORE: Execute the trigger body before the triggering DML event on a table
- AFTER: Execute the trigger body after the triggering DML event on a table
- INSTEAD OF: Execute the trigger body instead of the triggering DML event on a view
- Programming requirements will dictate which one will be used

ORACLE Academy

BEFORE triggers are often used to decide whether the triggering DML statement should be allowed to complete.

AFTER triggers are frequently used to generate audit records and log records.

INSTEAD OF triggers are designed to be used with complex views, not directly with tables.

If a trigger throws an unhandled error, the trigger, plus its triggering SQL statement would be rolled back. This would happen regardless of whether a BEFORE or AFTER trigger is used.

Trigger Timings and Events Examples

- The first trigger executes immediately before an employee's salary is updated:

```
CREATE OR REPLACE TRIGGER sal_upd_trigg
BEFORE UPDATE OF salary ON employees
BEGIN ... END;
```

- The second trigger executes immediately after an employee is deleted:

```
CREATE OR REPLACE TRIGGER emp_del_trigg
AFTER DELETE ON employees
BEGIN ... END;
```

Trigger Timings and Events Examples

- You can restrict an UPDATE trigger to updates of a specific column or columns:

```
CREATE OR REPLACE TRIGGER sal_upd_trigg
BEFORE UPDATE OF salary, commission_pct ON employees
BEGIN ... END;
```

- A trigger can have more than one triggering event:

```
CREATE OR REPLACE TRIGGER emp_del_trigg
AFTER INSERT OR DELETE OR UPDATE ON employees
BEGIN ... END;
```


In the first example, the trigger will fire if the SALARY or COMMISSION_PCT (or both) columns are updated, but not if other columns of EMPLOYEES are updated.

In the second example, the trigger will fire after any kind of DML statement is executed on EMPLOYEES. We could create three separate triggers (one for INSERT, one for UPDATE, one for DELETE), but a single trigger is neater.

How Often Does a Statement Trigger Fire?

- A statement trigger:
  - Fires only once for each execution of the triggering statement (even if no rows are affected)
  - Is the default type of DML trigger
  - Fires once even if no rows are affected
  - Useful if the trigger body does not need to process column values from affected rows

```
CREATE OR REPLACE TRIGGER log_emp_changes
AFTER UPDATE ON employees BEGIN
  INSERT INTO log_emp_table (who, when)
  VALUES (USER, SYSDATE);
END;
```

How Often Does a Statement Trigger Fire?

- Now an UPDATE statement is executed:

```
UPDATE employees
SET ... WHERE ...;
```

- How many times does the trigger fire, if the UPDATE statement modifies three rows?
- Ten rows?
- One row?
- No rows?

ORACLE
Academy

ANSWER: In every case, the trigger fires exactly once. A statement trigger fires only once even if the triggering DML statement affects many rows.

And When Does the Statement Trigger Fire?

- This slide shows the firing sequence for a statement trigger associated with the event INSERT INTO departments:

```
INSERT INTO departments
  (department_id,department_name, location_id)
VALUES (400, 'CONSULTING', 2500);
```

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID
10	Administration	1700
20	Marketing	1800
50	Shipping	1500
400	CONSULTING	2500

→ **BEFORE statement trigger** (before the table)

→ **AFTER statement trigger** (after the new row 400)

Trigger-Firing Sequence

- A statement trigger fires only once even if the triggering DML statement affects many rows:

```
UPDATE employees
  SET salary = salary * 1.1
  WHERE department_id = 50;
```

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
124	Mourgos	50
141	Rajs	50
142	Davies	50
143	Matos	50
144	Vargas	50

→ **BEFORE statement trigger**

→ **AFTER statement trigger**

ORACLE Academy

DML Statement Triggers Example 1

- This statement trigger automatically inserts a row into a logging table every time one or more rows are successfully inserted into EMPLOYEES

Application

```
INSERT INTO EMPLOYEES...;
```

EMPLOYEES table

EMPLOYEE_ID	LAST_NAME	JOB_ID
102	De Haan	AD_VP
100	King	AD_PRES
101	Kochhar	AD_VP

LOG_EMP trigger

```
CREATE OR REPLACE TRIGGER log_emp
AFTER INSERT ON employees BEGIN
  INSERT INTO log_emp_table (who, when)
  VALUES (USER, SYSDATE);
END;
```

ORACLE
Academy

Why is the trigger created as an AFTER trigger, not a BEFORE trigger?

ANSWER: Because we want to log the INSERT only if it is successful. The INSERT into EMPLOYEES could fail (for example because of a constraint violation), but a BEFORE trigger would already have logged the İNSERT by that time.

NOTE: This is a statement trigger, not a row trigger, and will therefore insert only one row into the logging table EVEN IF many employees are inserted, for example by INSERT….AS SELECT. In real life, a row trigger would be better here. Row triggers will be discussed in the next lesson.

DML Statement Triggers Example 2

- This statement trigger automatically inserts a row into a logging table every time a DML operation is successfully executed on the DEPARTMENTS table

```
CREATE OR REPLACE TRIGGER log_dept_changes
AFTER INSERT OR UPDATE OR DELETE ON DEPARTMENTS
BEGIN
  INSERT INTO log_dept_table (which_user, when_done)
  VALUES (USER, SYSDATE);
END;
```

DML Statement Triggers Example 3

- This example shows how you can use a DML trigger to enforce complex business rules that cannot be enforced by a constraint
- You want to allow INSERTs into the EMPLOYEES table during normal working days (Monday through Friday), but prevent INSERTs on the weekend (Saturday and Sunday)

A DML trigger used to enforce business rules and prevent unauthorized operations is sometimes called a Check Trigger.

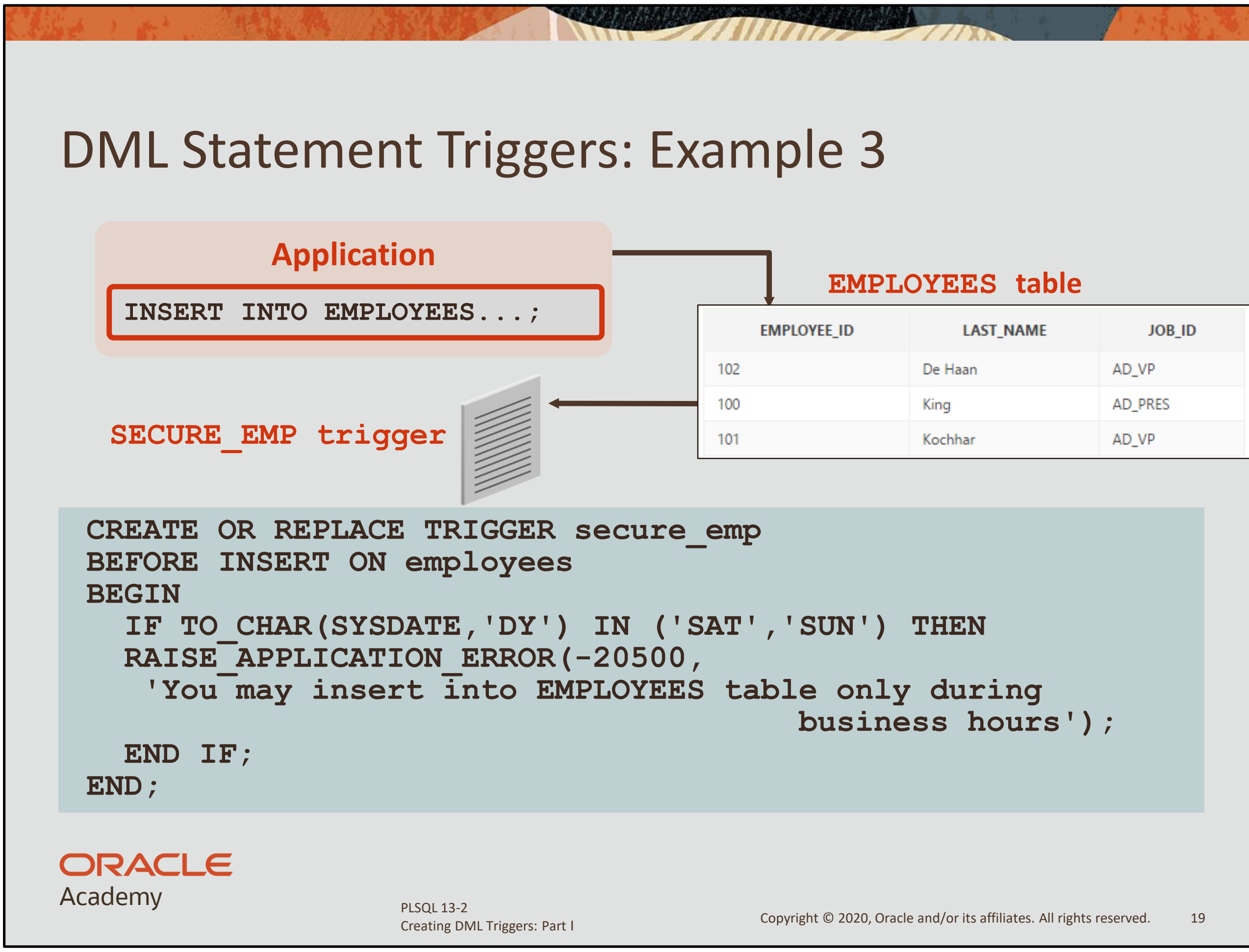
DML Statement Triggers Example

- If a user attempts to insert a row into the EMPLOYEES table during the weekend, then the user sees an error message, the trigger fails, and the triggering statement is rolled back
- The next slide shows the trigger code needed for this example

ORACLE
Academy

Remember the RAISE_APPLICATION_ERROR is a server-side, built-in procedure that returns an unhandled exception to the user and causes the PL/SQL block (in this case, the trigger) to fail.

When a database trigger fails, the triggering statement is automatically rolled back by the Oracle server. In other words, the employee is not inserted.



Why is this a BEFORE trigger?

ANSWER: If it were an AFTER trigger, then by the time the trigger raised the error, it would be too late—the employee would already have been inserted.

Testing SECURE_EMP

- A user tries to INSERT a row on the weekend:

```
INSERT INTO employees (employee_id, last_name, first_name,
  email, hire_date, job_id, salary, department_id)
  VALUES (300, 'Smith', 'Rob', 'RSMITH', SYSDATE,'IT_PROG',
  4500, 60);
```

```
ORA-20500:  You may insert into EMPLOYEES table only
  during business hours.
ORA-06512:  at "USVA_TEST_SQL01_T01.SECURE_EMP", line 4
ORA_04088:  error during execution of trigger
'USVA_TEST_SQL01_T01.SECURE_EMP' 2.  VALUES (300,
'Smith', 'Rob', 'RSMITH', SYSDATE, 'IT_PROG', 4500, 60);
```

ORACLE
Academy

You can demonstrate this concept by modifying the IF statement in the trigger to correspond with the current time. For example if you want to demonstrate this at 3:00 pm (15 on a 24 hour clock), recreate the trigger as:

```
CREATE OR REPLACE TRIGGER secure_emp
BEFORE INSERT ON employees
BEGIN
 IF TO_CHAR(SYSDATE,'HH24') BETWEEN 14 AND 16 THEN
 RAISE_APPLICATION_ERROR(-20500,
  'You may insert into EMPLOYEES table only during business hours');
 END IF;
END;
```

Before doing this, remember that SYSDATE returns the database time, and the database may be in a different time zone from you. Check the database time by:

```
SELECT TO_CHAR(SYSDATE, 'HH24:MI:SS') FROM DUAL;
```

A Final Example

- This trigger does not compile successfully
- Why not?

```
CREATE OR REPLACE TRIGGER log_dept_changes
AFTER INSERT OR UPDATE OR DELETE ON DEPARTMENTS
BEGIN
  INSERT INTO log_dept_table (which_user, when_done)
  VALUES (USER, SYSDATE);
  COMMIT;
END;
```

ORACLE
Academy

NOTE: Transaction control statements (COMMIT, ROLLBACK, SAVEPOINT) are not allowed in a trigger body.

Terminology

- Key terms used in this lesson included:
  - DML trigger
  - Row trigger
  - Statement trigger

ORACLE Academy

DML trigger – A trigger which is automatically fired (executed) whenever a SQL DML statement (INSERT, UPDATE or DELETE) is executed.

Row trigger – fires once for each row affected by the triggering event.

Statement trigger – is fired once on behalf of the triggering event, regardless of how many rows, or if any, are affected by the event.

Summary

- In this lesson, you should have learned how to:
  - Create a DML trigger
  - List the DML trigger components
  - Create a statement-level trigger
  - Describe the trigger firing sequence options

ORACLE
Academy

ORACLE
Academy